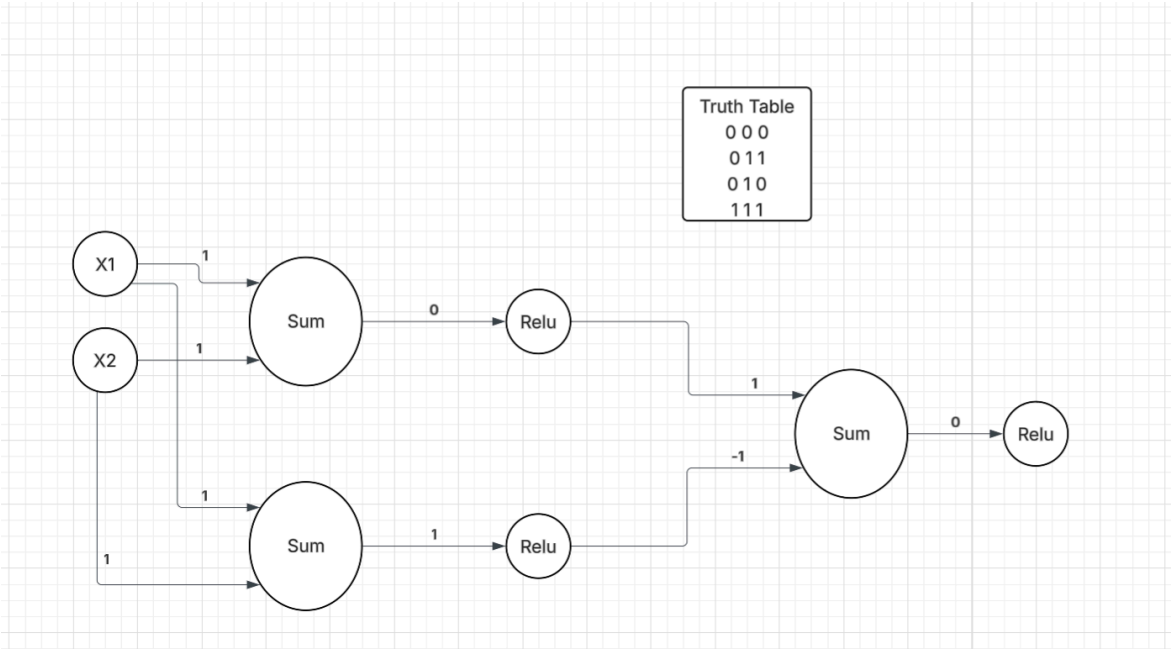
Stephen Hales
CS 6480

Midterm

For Part 1, I manually guessed the parameters for an OR gate model using a two-layer network with ReLU activations. Since a single ReLU neuron cannot represent the OR function (it needs to output 0 for (0,0) but 1 for all other inputs), I used two hidden neurons feeding into one output neuron. The first hidden neuron uses weights [1, 1] with bias 0, computing x1 + x2, which outputs a positive value whenever either input is 1. The second hidden neuron uses the same weights [1, 1] but with bias -1, computing x1 + x2 - 1, which only activates when both inputs are 1. The output neuron then takes the difference of these two hidden outputs with weights [1, -1] and bias 0. This produces 0 for (0,0), 1 for (0,1) and (1,0), and 1 for (1,1), matching the OR truth table exactly.



For Part 2, I used PyTorch to estimate the model parameters through gradient descent. I modeled the OR gate as a single-layer perceptron with two input weights and one bias term. The weights and bias were randomly initialized using a fixed seed of 42 for reproducibility. I used mean squared error (MSE) as the loss function and trained the model for 10,000 epochs with a learning rate of 0.5. At each epoch, I performed a forward pass to compute predictions, calculated the loss, then used PyTorch's autograd to compute gradients via backpropagation. The weights and bias were then updated manually using standard gradient descent (subtracting the learning rate times the gradient), and the gradients were zeroed before the next iteration.

I first tried using a ReLU activation function to match the manual approach from Part 1, but with a single neuron the results were poor. The final predictions were approximately 0.25, 0.75, 0.75,

and 1.25 for inputs (0,0), (0,1), (1,0), and (1,1). ReLU is unbounded above and linear for positive values, so a single ReLU neuron can only learn a linear function of the inputs. This means it cannot independently control the output for each input combination - it overshoots on (1,1) to 1.25 while undershooting on (0,1) and (1,0) to 0.75, and produces 0.25 instead of 0 for (0,0). As shown in Part 1, correctly implementing OR with ReLU requires multiple neurons across two layers.

I switched to a sigmoid activation function, which naturally bounds the output between 0 and 1, making it well-suited for learning binary logic gates with a single neuron. With sigmoid, the loss decreased steadily from 0.18 to 0.0005 over the course of training. The final predictions closely approximate the OR truth table, with outputs of approximately 0.03, 0.98, 0.98, and 1.00 for inputs (0,0), (0,1), (1,0), and (1,1) respectively.

Here is my code included below:

```
import torch
import torch.nn.functional as F

def compute_loss(prediction, y):
   return F.mse_loss(prediction, y)

def forward(X, weights, bias):
   return torch.sigmoid(X @ weights + bias)

X = torch.tensor([[0, 0],
                  [0, 1],
                  [1, 0],
                  [1, 1]], dtype=torch.float32)

y = torch.tensor([[0],
                  [1],
                  [1],
                  [1]], dtype=torch.float32)

torch.manual_seed(42)

weights = torch.randn(2, 1, requires_grad=True)  # 2 inputs, 1 output
bias = torch.randn(1, requires_grad=True)

learning_rate = 0.5
num_epochs = 10000
```

```
for epoch in range(num_epochs):
    prediction = forward(X, weights, bias)
    loss = compute_loss(prediction, y)
    # compute gradients
    loss.backward()

    with torch.no_grad():
        weights -= learning_rate * weights.grad
        bias -= learning_rate * bias.grad

    weights.grad.zero_()
    bias.grad.zero_()

    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")


print("\nFinal predictions:")
with torch.no_grad():
    final_pred = forward(X, weights, bias)
    print(final_pred)
print("\nExpected:")
print(y)
```